BELAJAR / MEMBANGUN / MENJELASKAN

Membuat GlowBook dari Awal

Panduan Ruby on Rails untuk pemula dengan contoh kode, alur fitur dan latihan

33 bab + 18 latihan

Baca satu bab, kerjakan satu perubahan kecil, lalu jelaskan alasan kode Anda. Atlas HTML mendampingi buku ini dengan sumber project per file dan nomor baris.

Versi belajar: 14 Oktober 2026
Snapshot kode: cb1cc60
Bahasa: Indonesia

Daftar isi

01 - Mulai dari sini 4

02 - Apa yang dibangun? 5

03 - Teknologi yang digunakan 6

04 - Cara browser berbicara dengan server 8

05 - Mengenal folder project 9

06 - Dasar Ruby dalam 15 menit 10

07 - Menyiapkan lingkungan belajar 12

08 - Urutan membangun dari nol 14

09 - MVC: model, view dan controller 15

10 - Database dan hubungan data 17

11 - Login dan pembatasan akses 18

12 - Marketplace dan pencarian 19

13 - Jadwal dan ketersediaan staff 20

14 - Alur booking lima langkah 21

15 - Pembatalan dan ubah jadwal 22

16 - Onboarding bisnis dan galeri 23

17 - Email, worker dan notifikasi 24

18 - Chat tanpa WebSocket 25

19 - Tabel, pencarian, filter dan jumlah data 26

20 - Tampilan rapi dengan Tailwind dan CSS 27

21 - Terjemahan dan PWA 28

22 - Integrasi Google, Facebook dan konfigurasi terenkripsi 29

23 - Duitku: invoice, callback dan status 30

24 - Laporan, analitik dan CSV 31

25 - Pengujian: bukti, bukan dugaan 32

26 - Docker, deployment dan backup 33

27 - Git dan pekerjaan sehari-hari 34

28 - Debugging tanpa panik 35

29 - Menjadi siap bekerja remote di Upwork 36

30 - Rencana belajar delapan tahap 37

31 - Cara membaca atlas kode 38

32 - Kamus istilah singkat 39

33 - Referensi dan dokumen lanjutan 40

Lampiran - Latihan dan Pembahasan 41

01 - Mulai dari sini

Untuk: pemula yang ingin memahami dan membangun aplikasi booking, lalu mampu menjelaskan pekerjaannya kepada klien remote.

Tanggal penulisan: 14 Oktober 2026. **Kode acuan:** commit `cb1cc60`. Dokumentasi ini menjelaskan implementasi project pada snapshot tersebut. Versi paket mengikuti `Gemfile.lock`, bukan versi terbaru di internet.

Tujuan Anda bukan menghafal semua baris. Tujuannya: mengetahui file mana yang perlu diubah, mengapa perubahan diperlukan, dan bagaimana membuktikan hasilnya.

Paket belajar ini berisi:

1. **PANDUAN_GLOWBOOK.html** - buku utama, nyaman dibaca offline, dengan daftar isi dan pencarian browser.

2. **PANDUAN_GLOWBOOK.md** - sumber buku yang dapat diedit.

3. **ATLAS_KODE.html** - kode project per file, nomor baris, fungsi file dan petunjuk membaca. Gunakan pencarian file untuk mencari `booking_builder` atau `users_controller`.

4. **LATIHAN_DAN_JAWABAN.md** - latihan, kriteria selesai, dan pembahasan.

Cara memakai: baca satu bab, ketik ulang contoh pada project latihan, jelaskan dengan kata sendiri, lalu lihat implementasi GlowBook di atlas. Jangan menyalin semua kode sekaligus.

Dua jalur yang berbeda

Jalur	Tujuan	Tempat mengerjakan
Menjalankan GlowBook yang sudah ada	Mengenali aplikasi dan struktur kode	Salinan repository dengan database latihan
Membangun ulang dari nol	Memahami setiap lapisan secara bertahap	Folder baru `glowbook_latihan`

Contoh yang berlabel **LATIHAN** adalah kode kecil untuk memahami konsep. Contoh yang berlabel **KODE PROJECT** berasal dari project ini. Contoh latihan bukan pengganti seluruh fitur keamanan GlowBook.

Jangan menjalankan reset/seed latihan pada database project yang berisi data penting. `db/seeds.rb` project menghapus data demo development sebelum mengisi ulang dan menolak berjalan di production.

02 - Apa yang dibangun?

GlowBook adalah marketplace layanan kecantikan dan wellness. Pelanggan mencari bisnis, memilih layanan dan staff, lalu memesan waktu. Owner mengatur bisnis. Admin mengelola platform.

Bagian	Contoh tugas pengguna	Lokasi utama
Publik	Cari salon, lihat layanan, jam buka dan ulasan	`/`, `/cari`, `/b/:slug`
Pelanggan	Booking, favorit, ulasan, profil, pesan	`/akun`
Owner/staff	Layanan, jadwal, booking, galeri, pelanggan	`/bisnis`
Admin	Pengguna, bisnis, kategori, promosi, laporan, integrasi	`/admin`
Integrasi	Google, Facebook, SMTP, Duitku	`/admin/integrasi`

Empat nilai peran di kode: `customer`, `business_owner`, `staff`, dan `admin`. Tampilan sehari-hari sering dikelompokkan menjadi tiga area: pelanggan, bisnis dan admin.

Yang sudah diimplementasikan tidak otomatis sudah aktif pada layanan nyata. Login Google memerlukan kredensial Google; email memerlukan SMTP; pembayaran memerlukan merchant Duitku dan callback publik.

Refund otomatis, payout owner, komisi otomatis dan WhatsApp belum tersedia. Angka pendapatan booking dan status pembayaran bukan bukti saldo bank. Ini penting saat menjelaskan cakupan kepada klien.

03 - Teknologi yang digunakan

Teknologi	Versi acuan / konfigurasi	Fungsi sederhana
Ruby	3.4.7	Bahasa backend: menghitung, memvalidasi dan mengolah data
Rails	8.1.3.1	Kerangka aplikasi: routes, controller, model, view, email, upload
PostgreSQL	Image Docker 16	Database relasional dan pencegahan bentrok booking
Active Record	Bagian Rails	Mengakses database memakai objek Ruby
HTML + ERB	Template Rails	Membentuk halaman HTML dengan data dari Ruby
Tailwind CSS	Gem 4.6.0; compiler sesuai lockfile	Mengatur warna, jarak, ukuran dan layout
Turbo	turbo-rails 2.0.23	Navigasi dan pembaruan HTML tanpa membuat SPA penuh
Stimulus	stimulus-rails 1.3.4	Perilaku kecil: menu, tutup pesan, geolokasi, chat
Importmap	2.2.3	Memetakan import JavaScript; tidak wajib memakai npm untuk runtime frontend ini
Propshaft	1.3.2	Menyajikan aset dengan fingerprint untuk cache
Devise	5.0.4	Login email/password, session dan reset password
OmniAuth	Facebook + Google OAuth2	Login memakai identitas penyedia eksternal
Active Storage	Bagian Rails	Relasi database dan penyimpanan file upload
libvips + image_processing	Library OS + gem	Memproses gambar dan variant
Action Mailer + Active Job	Bagian Rails	Menyusun email dan mengantrekan pengiriman
Solid Queue	1.7.0	Antrean pekerjaan menggunakan database; tidak memakai Redis pada konfigurasi ini
Puma	8.0.2	Server HTTP yang menjalankan Rails

Teknologi	Versi acuan / konfigurasi	Fungsi sederhana
Duitku POP	API eksternal	Hosted checkout dan status pembayaran
I18n + rails-i18n	Konfigurasi `id` dan `en`	Terjemahan dan format tanggal
RSpec + FactoryBot	rspec-rails 8.0.4	Pengujian otomatis Ruby dengan data contoh
Node test runner	`spec/javascript/*_test.cjs`	Menguji perilaku JavaScript; Node bukan server backend GlowBook
Brakeman	Gem development/test	Pemeriksaan potensi masalah keamanan secara statis
Docker + Compose	Dockerfile dan YAML	Menjalankan web, worker dan PostgreSQL secara konsisten
Git + GitHub Actions	Workflow repository	Riwayat perubahan dan pemeriksaan CI
Caddy	Overlay production	Reverse proxy dan HTTPS untuk deployment satu host

Gem yang terpasang tidak selalu berarti fitur aktif. Misalnya Jbuilder tersedia, tetapi respons chat saat ini dibuat dengan `render json:`. Capybara/Selenium tersedia sebagai alat uji; jangan menyebut semua alur otomatis memakai browser tanpa membaca spec.

Project ini tidak memakai React, Next.js, Laravel atau Node sebagai backend. Anda cukup belajar stack yang benar-benar dipakai dulu.

04 - Cara browser berbicara dengan server

Bayangkan satu tindakan: membuka daftar pengguna admin.

```
Browser: GET /admin/pengguna
  -> config/routes.rb memilih Admin::UsersController#index
  -> require_admin! memeriksa akses
  -> controller mengambil User dari PostgreSQL
  -> Paginatable menambahkan pencarian/filter/batas baris
  -> app/views/admin/users/index.html.erb membentuk HTML
  -> browser menampilkan tabel
```

Request adalah permintaan browser. **Response** adalah jawaban server. HTML dibentuk di server, kemudian JavaScript membantu interaksi tertentu.

Metode HTTP	Maksud	Contoh
GET	Membaca	Buka daftar atau form
POST	Membuat / tindakan server tertentu	Simpan pengguna baru, buat invoice
PATCH	Mengubah sebagian data	Edit pengguna, ubah status booking
DELETE	Menghapus	Hapus pengguna yang boleh dihapus

Status umum: `200` berhasil, `302/303` berpindah halaman, `404` tidak ditemukan, `422` input tidak valid, `503` layanan sementara tidak tersedia. Status HTTP berbeda dari status booking dan status Duitku.

05 - Mengenal folder project

Folder/file	Pertanyaan yang dijawab
`Gemfile` / `Gemfile.lock`	Library apa dan versi tepat apa yang digunakan?
`config/routes.rb`	URL ini masuk ke controller mana?
`app/controllers`	Bagaimana request dibaca dan respons dipilih?
`app/models`	Apa struktur relasi dan aturan validasi data?
`app/services`	Di mana proses bisnis yang panjang dilakukan?
`app/views`	Bagaimana halaman dan email disusun?
`app/helpers`	Bagaimana format rupiah, avatar, ikon dan teks dibantu?
`app/javascript/controllers`	Apa perilaku interaktif di browser?
`app/assets/tailwind/application.css`	Bagaimana komponen visual diatur?
`config/locales`	Dari mana teks Indonesia/Inggris berasal?
`db/migrate`	Bagaimana struktur database berubah secara bertahap?
`db/schema.rb`	Seperti apa hasil struktur database sekarang?
`app/mailers` / `app/jobs`	Bagaimana email disusun dan dijalankan di belakang?
`spec`	Bagaimana perilaku kode dibuktikan?
`public`	Ikon dan fallback offline yang dapat disajikan langsung
`compose.yaml` / `Dockerfile`	Bagaimana layanan dijalankan dalam container?
`script` / `deploy`	Pemeriksaan rilis, backup, restore dan HTTPS

Jangan mengedit `db/schema.rb` untuk menambahkan kolom; buat migration. Jangan mengedit CSS hasil build; edit sumber Tailwind. Jangan membuka `.env.local` di screenshot, menyalin secret ke tutorial, atau memasukkannya ke Git.

06 - Dasar Ruby dalam 15 menit

LATIHAN - kode ini dapat diketik pada `irb` di lingkungan Ruby latihan.

```
nama = "Facial"                # String: teks
harga = 150_000                 # Integer: bilangan bulat
aktif = true                    # Boolean: benar/salah
layanan = ["Facial", "Massage"]  # Array: daftar
booking = { nama: nama, harga: harga } # Hash: pasangan kunci/nilai

if aktif
  puts "#{nama}: Rp #{harga}"    # Interpolasi memasukkan nilai ke teks
end
```

`nama:` adalah key berupa Symbol. `booking[:harga]` membaca nilainya. Hash dengan key String menggunakan `booking["harga"]`; String dan Symbol bukan key yang sama.

```
def total_harga(harga, jumlah: 1)
  harga * jumlah
end

total_harga(150_000, jumlah: 2) # menghasilkan 300000
```

`def` membuat method. Nilai ekspresi terakhir menjadi hasil method. `jumlah:` adalah keyword argument. Pada service GlowBook, keyword seperti `business:` membuat pemanggilan mudah dibaca.

```
class KalkulatorBooking
  def initialize(harga:)
    @harga = harga
  end

  def total
    @harga
  end
end

KalkulatorBooking.new(harga: 150_000).total
```

`class` adalah rancangan objek. `.new` membuat objek. `initialize` menerima data awal. `@harga` adalah instance variable yang dapat dipakai method lain dalam objek yang sama.

Sintaks	Arti saat membaca GlowBook
`nil`	Tidak ada nilai
`present?` / `blank?`	Pemeriksaan nilai tersedia/kosong dari Active Support
`&.`	Panggil method hanya jika objek bukan nil
`||` / `&&`	Atau / dan
`return`	Keluar dari method dengan hasil tertentu
`.map` / `.select`	Ubah setiap elemen / pilih elemen yang sesuai
`.any?` / `.all?`	Apakah ada / apakah semuanya memenuhi kondisi?
`do ... end` / `{ ... }`	Block yang dijalankan suatu method
`private`	Method internal, bukan untuk dipanggil bebas dari luar

Sintaks	Arti saat membaca GlowBook
`rescue`	Tangani exception tertentu
`.freeze`	Cegah objek diubah setelah dibuat
`15.minutes`	Durasi dari Active Support, bukan Ruby dasar

Tanda `?` dan `!` di nama method adalah konvensi, bukan perintah khusus bahasa. Pada Active Record, `save!` menaikkan exception jika gagal, sedangkan `save` mengembalikan false. Method custom tetap perlu dibaca untuk mengetahui perilakunya.

07 - Menyiapkan lingkungan belajar

Untuk Windows, gunakan WSL2 Ubuntu untuk menulis Rails secara langsung, atau Docker Desktop untuk menjalankan project yang sudah ada. Jangan mencampur perintah Bash dengan PowerShell tanpa memperhatikan terminalnya.

Jalur A: project latihan baru dengan WSL/Ubuntu

Gunakan Ruby **3.4.7**, Rails **8.1.3.1**, PostgreSQL dan library OS. Detail pemasangan Ruby melalui rbenv ada di `SETUP.md`. Jalankan berikut hanya pada lingkungan latihan baru:

```
# Terminal WSL/Ubuntu, bukan PowerShell
sudo apt update
sudo apt install -y postgresql libpq-dev libvips build-essential libyaml-dev
ruby -v
gem install rails -v 8.1.3.1
rails new glowbook_latihan --database=postgresql --css=tailwind
cd glowbook_latihan
bundle install
```

Arti: `rails new` membuat skeleton, `--database` memilih PostgreSQL, `--css` memasang Tailwind, `bundle install` memasang gem. Ini belum menghasilkan seluruh GlowBook.

Buat role PostgreSQL latihan secara interaktif, agar password tidak ditulis di history shell:

```
sudo service postgresql start
sudo -u postgres createuser --login --createdb --pwprompt glowbook_latihan
read -s -p "Password database latihan: " PGPASSWORD
echo
export PGPASSWORD
export PGUSER=glowbook_latihan
export PGHOST=localhost
```

`read -s` menyembunyikan input. Role dan database latihan harus terpisah dari GlowBook utama. Sesuaikan bagian development/test `config/database.yml` project latihan:

```
default: &default
  adapter: postgresql
  encoding: unicode
  host: <%= ENV.fetch("PGHOST", "localhost") %>
  username: <%= ENV.fetch("PGUSER", "glowbook_latihan") %>
  password: <%= ENV["PGPASSWORD"] %>
  pool: 5

development:
  <<: *default
  database: glowbook_latihan_development

test:
  <<: *default
  database: glowbook_latihan_test
```

`&default` membuat anchor YAML, `<<: *default` menggunakan kembali konfigurasi. `<%= ... %>` menghitung nilai Ruby sebelum YAML dibaca. Simpan password di environment, bukan pada file tersebut.

```
bin/rails db:create
bin/rails db:migrate
bin/dev
```

`db:create` membuat database; `db:migrate` menjalankan migration; `bin/dev` menjalankan proses development sesuai `Procfile.dev`, termasuk watcher CSS. Buka `http://localhost:3000`. Jika GlowBook utama memakai port 3000, hentikan server latihan dan atur port latihan tersendiri, misalnya server Rails port 3002 dengan watcher

Tailwind di terminal kedua.

Jalur B: menjalankan repository GlowBook

`SETUP.md` adalah petunjuk operasional project. Clone ke folder baru jika ingin bereksperimen. `bundle install`, `bin/rails db:prepare`, lalu `bin/dev` menjalankan mode development. Perhatikan bahwa prepare dapat memuat seed pada database development baru.

Stack Docker lokal yang sudah ada memakai konfigurasi production dengan HTTP lokal yang diizinkan khusus di `.env.local`. Untuk menyalakan ulang stack tersebut, dari PowerShell di folder GlowBook:

```
docker compose --env-file .env.local up -d --wait web worker
docker compose --env-file .env.local ps
docker compose --env-file .env.local logs --tail 50 web
```

`up` menjalankan layanan, `-d` di belakang, `--wait` menunggu healthcheck, `ps` menunjukkan status, `logs` membaca keluaran. Perintah ini memakai container yang sudah ada; bukan langkah membuat project baru. Port default Compose adalah **3001**, sedangkan konfigurasi `.env.local` project memakai **3000**.

Setelah mengubah kode, image production harus dibangun ulang agar perubahan masuk. Development biasanya melakukan reload; container production tidak otomatis membaca file kerja host.

08 - Urutan membangun dari nol

Jangan langsung mulai dari payment gateway. Bangun satu alur yang dapat dipakai, kemudian tambah lapisan.

Tahap	Yang dibuat	Bukti selesai
1	Homepage statis, layout dan navbar	Dapat dibuka dengan tampilan rapi
2	Model kota, kategori, bisnis, layanan	Data dapat dibuat dan ditampilkan
3	Devise dan pembatasan peran	Customer tidak dapat membuka admin
4	CRUD layanan owner	Owner hanya mengubah layanannya sendiri
5	Staff, kualifikasi dan jadwal	Slot mempertimbangkan jam kerja dan istirahat
6	Booking bertahap dan constraint DB	Dua request tidak memesan interval staff yang sama
7	Akun pelanggan, favorit dan ulasan	Riwayat dan akses milik pengguna benar
8	Onboarding, galeri dan admin	Bisnis pending, upload tersimpan, moderasi tersedia
9	Notifikasi, email dan chat	Pesan/email mencapai peserta yang benar
10	Filter, pagination, analitik dan CSV	Hasil sesuai filter dan tidak bocor antar owner
11	I18n, PWA dan keamanan	Bahasa bekerja dan halaman privat tidak disimpan offline
12	Integrasi, deployment dan backup	Kredensial nyata diverifikasi; restore dibuktikan

Pada tahap awal, gunakan satu kota, satu bisnis, dua layanan dan satu staff. Kompleksitas baru ditambah setelah contoh kecil dipahami.

09 - MVC: model, view dan controller

Model adalah data dan aturannya. **View** adalah tampilan. **Controller** menghubungkan request dengan model dan view. Service menampung proses yang terlalu panjang untuk controller.

LATIHAN - buat katalog layanan mini pada project latihan.

```
bin/rails generate model Service name:string price:integer duration_minutes:integer
  active:boolean
bin/rails db:migrate
bin/rails generate controller Services index new
```

Model generator menghasilkan model dan migration, bukan fitur booking. Ini memakai model `Service` mini tanpa bisnis dan kategori; GlowBook asli memiliki relasi tambahan.

Ganti routes hasil generator untuk controller latihan menjadi:

```
Rails.application.routes.draw do
  root "services#index"
  resources :services, only: [:index, :new, :create]
end
```

Ganti file model latihan `app/models/service.rb`:

```
class Service < ApplicationRecord
  validates :name, presence: true
  validates :price, numericality: { only_integer: true, greater_than_or_equal_to: 0 }
  validates :duration_minutes, numericality: { only_integer: true, greater_than: 0 }
end
```

`< ApplicationRecord` mewarisi Active Record. `validates` memeriksa data sebelum penyimpanan. Harga integer menyimpan rupiah tanpa persoalan pecahan float.

Ganti `app/controllers/services_controller.rb` pada latihan:

```
class ServicesController < ApplicationController
  def index
    @services = Service.order(:name)
  end

  def new
    @service = Service.new
  end

  def create
    @service = Service.new(service_params)
    if @service.save
      redirect_to services_path, notice: "Layanan berhasil dibuat."
    else
      render :new, status: :unprocessable_content
    end
  end

  private

  def service_params
    params.require(:service).permit(:name, :price, :duration_minutes)
  end
end
```

Baris/bagian	Penjelasan
`index`	Mengambil daftar; Rails mencari view `services/index`

Baris/bagian	Penjelasan
`@services`	Data dapat dibaca view karena instance variable
`Service.order(:name)`	Membentuk query urut nama
`new`	Menyiapkan objek kosong untuk form
`create`	Membaca input, membuat objek dan mencoba menyimpan
`redirect_to`	Membuat request baru menuju daftar setelah berhasil
`render :new`	Menampilkan form kembali tanpa request baru
`422`	Memberi tahu bahwa input belum valid
`require(:service)`	Memerlukan kelompok parameter service
`permit(...)`	Hanya menerima field yang diizinkan

Latihan ini belum dilindungi login. Setelah CRUD dipahami, tambahkan autentikasi dan ownership sebelum memakai pola ini untuk data klien.

`app/views/services/new.html.erb` latihan:

```
<h1>Tambah layanan</h1>
<% @service.errors.full_messages.each do |message| %>
  <p><%= message %></p>
<% end %>

<%= form_with model: @service do |f| %>
  <%= f.label :name, "Nama" %>
  <%= f.text_field :name, required: true %>
  <%= f.label :price, "Harga rupiah" %>
  <%= f.number_field :price, min: 0, required: true %>
  <%= f.label :duration_minutes, "Durasi menit" %>
  <%= f.number_field :duration_minutes, min: 1, required: true %>
  <%= f.submit "Simpan" %>
<% end %>
```

`<% ... %>` menjalankan Ruby; `<%= ... %>` menampilkan hasilnya. `f` adalah pembuat field form. `required/min` membantu browser, tetapi validasi server tetap wajib. `form_with` ikut menyiapkan token CSRF.

`app/views/services/index.html.erb` latihan:

```
<h1>Layanan</h1>
<%= link_to "Tambah layanan", new_service_path %>
<% @services.each do |service| %>
  <article>
    <h2><%= service.name %></h2>
    <p>Rp <%= service.price %> | <%= service.duration_minutes %> menit</p>
  </article>
<% end %>
```

Nama helper seperti `new_service_path` dibuat Rails dari routes. Setelah latihan berhasil, bandingkan dengan `Dashboard::ServicesController` dan form layanan GlowBook di atlas. Perbedaan penting: scope `@business.services`, kategori, terjemahan, staff, dan perlindungan layanan yang telah dipakai booking.

10 - Database dan hubungan data

```
User (owner) -> Business -> Service -> BookingItem <- Booking <- User (customer)
                    |         |            |
                    |         +-- StaffService -- Staff
                    +-- BusinessHour              |-- StaffSchedule
                    +-- Promotion                 |-- StaffBreak
                    +-- BusinessGallery           +-- BlockedTime
Booking -> PaymentAttempt
User + Business -> Favorite / Review / Conversation
Conversation -> ChatMessage
User -> UserNotification
```

Panah berarti hubungan; jumlah record tergantung relasinya. `BookingItem` adalah detail layanan dalam satu booking. Satu booking dapat berisi beberapa layanan berurutan dengan snapshot harga dan durasi.

KODE PROJECT - bagian model Service:

```
belongs_to :business
belongs_to :category
has_many :booking_items, dependent: :restrict_with_error
has_many :staff_services, dependent: :destroy
has_many :staff, through: :staff_services
```

`belongs_to` berarti record memiliki foreign key. `has_many ... through` membaca relasi melalui tabel penghubung. Staff dan layanan banyak-ke-banyak: satu staff dapat melayani banyak layanan dan satu layanan dapat dikerjakan banyak staff.

`restrict_with_error` menolak penghapusan layanan yang sudah memiliki booking item. `dependent: :destroy` menghapus record terkait ketika induk dihapus; karena itu tidak aman menghapus data bisnis bersejarah sembarangan.

Migration adalah riwayat perubahan skema. **LATIHAN** menambah kolom deskripsi:

```
bin/rails generate migration AddDescriptionToServices description:text
bin/rails db:migrate
```

Migration menambah struktur, sedangkan model mengatur perilaku. `db:migrate` tidak otomatis membuat form deskripsi. Anda masih perlu menambah permit controller dan field view.

`foreign_key: true` membantu DB memastikan induk tersedia. `unique: true` pada index melindungi keunikan ketika request bersamaan. Validasi Ruby saja tidak menggantikan constraint database.

11 - Login dan pembatasan akses

KODE PROJECT - User:

```
devise :database_authenticatable, :registerable, :recoverable, :rememberable,
       :validatable, :omniauthable, omniauth_providers: [ :facebook, :google_oauth2 ]
enum :role, { customer: 0, business_owner: 1, staff: 2, admin: 9 }, default: :customer
```

Devise mengelola autentikasi password, pendaftaran, reset, remember-me, validasi dan kaitan OAuth. Password disimpan sebagai hash melalui Devise, bukan sebagai teks biasa. Enum menyimpan angka di database dan memberi nama serta predicate seperti `admin?`.

Authentication bertanya "siapa Anda?". **Authorization** bertanya "apa yang boleh Anda lakukan?". Login saja belum menjamin boleh mengedit bisnis lain.

KODE PROJECT - pola akses:

```
before_action :require_admin!
# Di dashboard owner, pilih record melalui bisnis yang dapat diakses:
@service = @business.services.find(params[:id])
```

`before_action` berjalan sebelum action. Scope `@business.services` mencegah owner memilih layanan bisnis lain melalui ID. Jangan menggantinya dengan `Service.find(params[:id])` tanpa pemeriksaan ownership.

`ApplicationController#after_sign_in_path_for` memilih tujuan setelah login: admin, dashboard bisnis, atau akun pelanggan. `require_customer!` project saat ini hanya memastikan login; namanya tidak berarti ia menolak semua peran non-customer. Baca kondisi aktual, jangan menebak dari nama method.

Admin tambah/edit pengguna mempunyai strong parameters untuk profil/password dan menetapkan role secara eksplisit dari `User.roles`. Perubahan role akun admin, akun sendiri, owner dengan bisnis, atau staff terhubung ditolak. Email akun OAuth tidak dapat diganti dari panel tersebut.

Kolom password kosong saat edit dipisahkan dari atribut yang disimpan. Jika tidak, password kosong dapat merusak alur validasi atau membingungkan pengguna.

12 - Marketplace dan pencarian

`SearchesController#index` menyusun query dari bisnis aktif, lalu menambahkan kota, kata pencarian, rating, promo, jam buka, kategori dan harga. Query dibatasi 12 bisnis per halaman.

KODE PROJECT - contoh scope:

```
scope = Business.active.includes(:city, :categories, :services, :promotions)
scope = scope.in_city(params[:city]) if params[:city].present?
scope = scope.search(params[:q]) if params[:q].present?
```

`scope` adalah kumpulan record yang masih berupa relation. `where`/scope menyusun SQL; query biasanya dieksekusi ketika data dibaca. `includes` mengurangi query berulang untuk relasi yang akan ditampilkan.

`ILIKE` adalah pencarian PostgreSQL tanpa membedakan kapital. Binding seperti `where("name ILIKE ?", nilai)` memisahkan nilai dari SQL. Hindari menyisipkan input mentah langsung ke string SQL.

Geolokasi diproses `nearby_search_controller.js` dan endpoint POST `/cari/terdekat`. Koordinat diperiksa rentangnya, disimpan di session dengan masa berlaku 30 menit, dan digunakan untuk urutan jarak. Browser dapat menolak izin lokasi; aplikasi perlu tetap menyediakan pencarian biasa.

Homepage/detail bisnis menggunakan partial untuk kartu, galeri, jam dan ulasan. Partial berawalan _ membantu memakai tampilan yang sama di beberapa halaman.

13 - Jadwal dan ketersediaan staff

Slot tidak dibuat dengan menampilkan jam 09.00-17.00 saja. Slot harus sesuai jam bisnis, durasi layanan, staff aktif yang memenuhi kualifikasi, shift, break, blocked time dan booking sebelumnya.

KODE PROJECT - Availability:

```
SLOT_GRANULARITY = 30.minutes

def staff_free?(staff_id, start, finish)
  working_hours?(staff_id, start, finish) &&
    !on_break?(staff_id, start, finish) &&
    !blocked?(staff_id, start, finish) &&
    !booked?(staff_id, start, finish)
end
```

`30.minutes` adalah langkah pilihan waktu awal yang ditampilkan, bukan durasi wajib semua layanan. Layanan kedua dapat mulai pada menit di antara pilihan awal selama seluruh intervalnya valid.

Contoh: layanan A 45 menit dan B 30 menit mulai pukul 09.00. A menempati 09.00-09.45, B 09.45-10.15. Bila staff istirahat 10.00-10.30, rangkaian itu tidak tersedia.

`working_hours?` memeriksa apakah shift menutup seluruh interval. Shift berurutan dapat menutupi satu layanan; celah antarsift tidak boleh dilewati. Waktu di database menggunakan UTC, sementara perhitungan antarmuka memakai zona Jakarta. Zona komputer Anda bukan dasar perhitungan booking.

KODE PROJECT - overlap SQL:

```
.where("booking_items.during && tstzrange(?, ?, '[)')", start, finish)
```

`tstzrange` adalah rentang timestamp PostgreSQL. `&&` memeriksa tumpang tindih. `[)` memasukkan batas awal dan mengecualikan batas akhir: layanan selesai 10.00 boleh diikuti layanan mulai 10.00.

`BookingSequenceAvailability` menguji seluruh rangkaian layanan. `Availability` menguji kebutuhan interval layanan/staff. Keduanya perlu dibaca bersama untuk memahami booking beberapa layanan.

14 - Alur booking lima langkah

`Layanan -> Staff -> Tanggal -> Waktu -> Konfirmasi -> Simpan`

`BookingsFlowController` menyimpan pilihan sementara di `session[:booking_flow]`. Mengubah layanan menghapus pilihan staff/tanggal/waktu yang tidak lagi relevan. Business ID diperiksa agar session dari satu bisnis tidak digunakan pada bisnis lain.

State dari browser tetap diperiksa ulang di server. Harga berasal dari database, bukan hidden input pelanggan. Staff harus aktif, milik bisnis, dan dapat mengerjakan semua layanan yang dipilih.

KODE PROJECT - controller memanggil service:

```
result = BookingBuilder.new(
  user: current_user,
  business: @business,
  service_ids: flow["service_ids"],
  staff_id: flow["staff_id"],
  start_time: flow_start_time,
  payment_method: payment_method
).call
```

`.new` memasukkan kebutuhan proses; `.call` menjalankannya. `result.ok?` menunjukkan keberhasilan. Jika gagal, `result.error` menjadi pesan. Controller memilih respons; service menjalankan proses bisnis.

KODE PROJECT - transaksi di BookingBuilder:

```
Booking.transaction(requires_new: true) do
  booking = Booking.create!(...)
  services.each do |service|
    BookingItem.create!(...)
  end
  BookingResult.new(booking: booking.reload)
end
```

Tanda `...` pada potongan ini adalah penyingkat penjelasan, bukan kode runnable. Kode lengkap ada di atlas. Transaksi berarti booking dan item harus tersimpan sebagai satu kesatuan; exception membuat rollback. `reload` membaca ulang asosiasi setelah item dibuat.

Harga dan durasi disalin ke BookingItem. Bila harga layanan diubah besok, booking bersejarah tetap menyimpan harga yang dipakai saat dipesan.

Mengapa perlu perlindungan database?

Dua pelanggan dapat membaca slot kosong pada saat yang hampir sama. Pemeriksaan aplikasi saja tidak menutup perlombaan tersebut.

KODE PROJECT - migration:

```
EXCLUDE USING gist (
  staff_id WITH =,
  during WITH &&
) WHERE (during IS NOT NULL AND staff_id IS NOT NULL)
```

Constraint menolak interval yang bertabrakan pada staff yang sama. `btree_gist` digunakan untuk menggabungkan pembandingan staff dan range. BookingBuilder menangani exception dengan pesan slot sudah diambil.

Reference booking dibuat model dalam pola `GB-YYYYMMDD-000001`. Pembentukan urutan dengan membaca maksimum masih dapat bertabrakan saat request serentak; unique index melindungi database, tetapi tidak berarti semua request akan berhasil. Jangan mengklaim generator reference itu sequence database atomik.

15 - Pembatalan dan ubah jadwal

`Booking#cancel!` memakai `with_lock`, melepaskan interval item dengan `during: nil`, lalu mengubah status. `with_lock` mengunci record dalam transaksi agar dua perubahan pada booking yang sama tidak saling menimpa.

Aturan aktual project:

- Pembatalan hanya untuk pending/confirmed dan ditolak jika ada payment attempt pending/settlement/capture.

- Ubah jadwal memakai aturan pembatalan tersebut dan waktu awal booking harus lebih dari dua jam ke depan.

- Selesai hanya untuk confirmed setelah waktu berakhir.

- No-show hanya untuk confirmed setelah waktu mulai.

`BookingRescheduler` mengecek rangkaian layanan menggunakan harga/durasi booking yang tersimpan. Interval lama dilepas di dalam transaksi, interval baru ditulis, lalu waktu booking diperbarui. Jika bentrok, rollback mengembalikan data lama.

Setelah perubahan berhasil, `BookingNotificationDelivery` dipanggil untuk mengabari pelanggan dan owner. Pembayaran yang sudah diterima membutuhkan kebijakan refund tersendiri; project belum melakukan refund otomatis.

16 - Onboarding bisnis dan galeri

`BusinessRegistrationsController#create` membuat bisnis pending, memilih kategori yang valid, membuat jam buka awal dan menaikkan customer menjadi owner dalam transaksi. Pada onboarding pelanggan, jam default 09.00-17.00 selain Minggu; bisnis masih pending. Pada penambahan bisnis oleh admin, tujuh hari dimulai tertutup. Kedua alur berbeda, jadi periksa file masing-masing.

Owner melengkapi layanan, staff, jadwal dan profil. Admin menyetujui, menangguhkan atau mengaktifkan kembali. Menangguhkan bisnis mempertahankan booking dan riwayat.

KODE PROJECT - BusinessGallery:

```
belongs_to :business
has_one_attached :image
```

`has_one_attached` menghubungkan record dengan metadata dan blob Active Storage. File bukan disimpan sebagai string foto di tabel galeri.

Validasi galeri project menerima JPEG, PNG atau WebP, maksimum 10 MB per file; controller membatasi jumlah galeri enam. File owner diakses melalui `@business.business_galleries`, sehingga owner tidak dapat menghapus gambar bisnis lain lewat ID.

Production saat ini memakai disk lokal `/rails/storage` dengan volume Docker. Database menyimpan metadata, volume menyimpan byte gambar. Backup database saja tidak cukup. Bila pindah ke hosting dengan disk sementara, siapkan object storage atau disk permanen dan sesuaikan konfigurasi.

17 - Email, worker dan notifikasi

```
Booking berhasil
  -> BookingNotificationDelivery
  -> UserNotification dibuat untuk pelanggan dan owner
  -> BookingMailer menyiapkan email
  -> deliver_later memasukkan pekerjaan ke antrean
  -> worker Solid Queue menjalankan pengiriman SMTP
```

`deliver_later` tidak membuktikan email sudah sampai ke inbox. Ia mengantrekan pekerjaan. Worker harus berjalan, SMTP harus valid, dan penyedia email dapat menolak pesan.

KODE PROJECT - pemanggilan mailer:

```
BookingMailer.with(booking: @booking, recipient: recipient, locale: I18n.locale)
             .public_send(@event).deliver_later
```

`.with` memasukkan parameter. `@event` dibatasi ke confirmation/cancellation/reschedule sebelum `public_send` dipanggil. Jangan mengganti allowlist itu dengan nama method bebas dari input pengguna.

`ApplicationMailer` membaca konfigurasi SMTP tersimpan saat pengiriman. Google/Facebook/SMTP/Duitku di Admin disimpan terenkripsi melalui IntegrationSetting. SMTP dapat memakai STARTTLS atau TLS sesuai penyedia.

Production menggunakan dua database bernama `glowbook_production` dan `glowbook_production_queue` pada server PostgreSQL yang sama. Yang kedua menyimpan antrean. Backup perlu mencakup keduanya.

UserNotification memiliki read/unread; controller memilih record dari pengguna yang login. Menandai semua terbaca bukan menghapus notifikasi.

18 - Chat tanpa WebSocket

Chat GlowBook saat ini memakai polling HTTP tiap empat detik. Ia tidak memakai Action Cable atau Redis untuk pengiriman pesan real-time.

KODE PROJECT - Stimulus:

```
connect() {
  this.scrollToLatest()
  this.timer = window.setInterval(() => this.refresh(), 4000)
}

disconnect() {
  window.clearInterval(this.timer)
}
```

`connect` berjalan ketika controller dipasang pada elemen. Interval memanggil refresh. `disconnect` membersihkan timer saat elemen dilepas agar tidak menumpuk setelah navigasi Turbo.

Server menerima `after_id` dan mengirim pesan yang lebih baru. Pesan hanya boleh dibaca peserta percakapan. Owner, pelanggan dan staff aktif diperiksa model Conversation.

KODE PROJECT - render pesan aman:

```
body.textContent = message.body
```

`textContent` menampilkan pesan sebagai teks, bukan mengeksekusinya sebagai HTML. Jangan mengganti ini dengan `innerHTML` untuk pesan pengguna.

Chat menampilkan 100 pesan terakhir dan polling dibatasi 100 record per respons. Ini bukan fitur riwayat chat tanpa batas. Teks `Anda` di controller JavaScript masih hardcode; jangan menyebut seluruh kode sudah 100% terjemahan.

19 - Tabel, pencarian, filter dan jumlah data

`Paginatable` dipakai banyak controller. Ia mempertahankan scope milik pengguna/bisnis, menambahkan pencarian dan filter, menghitung hasil, lalu memberi limit/offset.

KODE PROJECT - batas baris:

```
scope.limit(@per_page).offset((@page - 1) * @per_page)
```

Jika 25 data per halaman: halaman 1 offset 0, halaman 2 offset 25, halaman 3 offset 50. Pilihan ukuran dibatasi 10/25/50/100. `all` menggunakan jumlah hasil, dengan minimum 1 untuk perhitungan halaman agar tidak membagi nol.

Permintaan filter disusun seperti:

```
tables[users][q]=sari
tables[users][per_page]=10
tables[users][role]=customer
```

Namespacing per tabel membuat filter memiliki identitas jelas. GET dipakai sehingga URL dapat disalin. Tombol Terapkan memulai halaman baru tanpa mempertahankan page lama; link pagination mempertahankan filter.

`table_search_predicate` memakai Arel untuk kondisi SQL. Ia mencari field teks yang dikenali dan dapat menjangkau relasi belongs_to sampai dua tingkat. Ini bukan mesin pencarian semua kolom; lihat daftar field aktual di concern.

`sanitize_sql_like` meloloskan `%`/_ dari kata pencarian agar dibaca sebagai karakter biasa. Filter role/status memakai enum; filter relasi memakai pilihan ID yang tersedia pada scope.

`table_date_column` memilih `start_time`, lalu `starts_at`, lalu `created_at`. Karena itu tanggal booking berarti jadwal, tanggal pengguna berarti dibuat, tanggal promosi berarti mulai berlaku. Penjelasan basis tanggal ditampilkan pada form.

Pencarian/CSV pembayaran mengikuti scope yang sama. Mode sandbox tidak ditambahkan ke total uang production. Pilihan Semua dapat berat pada data besar; tidak ada jaminan kapasitas ribuan record sebelum pengukuran.

20 - Tampilan rapi dengan Tailwind dan CSS

LATIHAN:

```
<section class="rounded-2xl bg-white p-6 shadow-sm">
  <h2 class="mb-4 text-xl font-bold">Filter pengguna</h2>
</section>
```

`rounded-2xl` membulatkan sudut; `bg-white` memberi latar; `p-6` padding; `shadow-sm` bayangan; `mb-4` jarak bawah; `text-xl` ukuran; `font-bold` ketebalan.

Jarak penting: **padding** adalah jarak isi dari tepi kotak; **margin** jarak antar kotak; **gap** jarak antar item grid/flex.

KODE PROJECT - perapian filter:

```
.table-controls {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 170px), 1fr));
  align-items: end;
  gap: 18px 16px;
  padding: 24px;
}
```

Grid membuat kolom menyesuaikan ruang. Minimum 170 px mencegah field terlalu sempit pada layar besar. `min(100%, ...)` membantu saat wadah lebih kecil. `align-items: end` menyelaraskan dasar field. Padding mengatasi teks yang sebelumnya menempel pada card.

Pada layar di bawah 640 px, project memakai dua kolom, padding 20 px vertikal dan 16 px horizontal. Pencarian, tombol, dan ringkasan memakai satu baris penuh. `min-width: 0` membantu item tidak memaksa grid melebar.

Untuk menambah komponen visual: lihat style yang sudah ada terlebih dahulu, gunakan class bersama, periksa label/keyboard/kontras dan layar kecil. Jangan langsung menambah JavaScript untuk persoalan yang cukup diselesaikan dengan CSS.

21 - Terjemahan dan PWA

KODE PROJECT - terjemahan:

```
<%= t("table_controls.search") %>
```

Key tersebut dibaca dari YAML locale. Tambahkan key pada `id` dan `en`, bukan mengetik label dua kali di view. `l(...)` memformat tanggal/waktu sesuai locale. Locale dipilih melalui session dan `I18n.with_locale`.

PWA terdiri dari manifest, ikon, service worker dan halaman offline. Worker menyimpan aset statis; halaman akun/admin tetap menggunakan jaringan dan tidak dicache sebagai halaman privat. Jika navigasi gagal, fallback offline ditampilkan.

KODE PROJECT - pembatasan interception:

```
if (request.method !== "GET" || url.origin !== self.location.origin) return
```

Worker tidak mengambil alih mutasi POST/PATCH/DELETE atau request domain lain. Booking dan pembayaran tetap membutuhkan server; PWA ini tidak memberi kemampuan booking offline.

Versi cache dinaikkan jika strategi atau aset berubah. Aktivasi membersihkan cache milik prefix GlowBook, bukan semua cache origin sembarangan.

22 - Integrasi Google, Facebook dan konfigurasi terenkripsi

OAuth mengalihkan pengguna ke penyedia, penyedia mengembalikan callback, lalu aplikasi membuat/mencari identitas yang cocok. `state`/CSRF melindungi proses. Callback URI harus sama persis dengan yang didaftarkan, termasuk port lokal.

Google mengharuskan email terverifikasi. Identitas dicari berdasarkan provider dan UID, bukan hanya email. Akun dengan email sama tidak otomatis digabung; ini mengurangi risiko pengambilalihan akun. Penautan beberapa identitas belum tersedia.

KODE PROJECT - penyimpanan integrasi:

```
key = Rails.application.key_generator.generate_key("glowbook-integration-settings-v1", 32)
ActiveSupport::MessageEncryptor.new(key, cipher: "aes-256-gcm", serializer: JSON)
```

Key diturunkan melalui key generator Rails. AES-256-GCM menyimpan isi terenkripsi dengan verifikasi. Purpose memisahkan konteks tiap provider. Jangan menciptakan algoritma enkripsi sendiri.

Konfigurasi tersimpan mengesampingkan fallback environment, termasuk nilai disabled. Kolom secret selalu kosong di form; kosong saat edit berarti mempertahankan secret lama, bukan menghapus. Penghapusan memerlukan pilihan khusus.

`SECRET_KEY_BASE` yang sama diperlukan agar secret dapat dibaca setelah restore. Database backup tanpa key tidak cukup; key perlu disimpan terpisah secara aman. Jangan mengunggah key ke GitHub atau atlas kode.

23 - Duitku: invoice, callback dan status

```
Customer menekan bayar
  -> CreateBookingPayment membuat/memakai attempt
  -> DuitkuClient meminta invoice
  -> Browser ke hosted checkout Duitku
  -> Duitku POST callback ke aplikasi
  -> Signature dan identitas diperiksa
  -> Server meminta transactionStatus Duitku
  -> PaymentAttempt diperbarui berdasarkan status API
```

Browser return URL bukan bukti pembayaran. Pengguna dapat mengubah query URL. Status lunas diputuskan setelah verifikasi server.

KODE PROJECT - signature:

```
OpenSSL::HMAC.hexdigest("SHA256", @api_key, value)
```

HMAC memakai secret bersama. String yang ditandatangani berbeda untuk create invoice, callback dan cek status. Lihat `DuitkuClient` serta `DUITKU_SETUP.md` untuk urutan field dan endpoint acuan. Jangan memakai contoh MD5 lama jika dokumentasi merchant Anda mengharuskan HMAC.

API key tidak dikirim sebagai parameter browser. Checkout URL harus HTTPS dan host Duitku yang diizinkan. Callback memeriksa merchant, mode, order, reference, jumlah dan signature. API status harus cocok dengan attempt tersimpan.

`CreateBookingPayment` menggunakan lock konfigurasi dan lock booking. Jika hasil invoice ambigu setelah timeout, order ID dipertahankan; membuat ID baru langsung dapat memicu pembayaran ganda.

`ReconcileDuitkuPayment` menangani status server, callback berulang dan refresh manual. Paid tidak boleh berubah kembali menjadi pending akibat callback terlambat. Ini disebut **idempotensi**: pengulangan tindakan tidak menggandakan efek yang sudah berhasil.

Sandbox ditandai pada riwayat dan tidak masuk total uang production. Record Midtrans lama dipertahankan, tetapi integrasi aktif checkout baru memakai Duitku.

Tes HTTP stub membuktikan aturan kode, bukan persetujuan merchant atau dana sampai bank. Uji nyata memerlukan kredensial sandbox, host callback publik dan simulator penyedia. Jangan menyalakan production sebelum verifikasi itu selesai.

24 - Laporan, analitik dan CSV

Controller laporan memakai periode bulan; grafik memakai layanan `AdminCharts`. Dashboard bisnis menghitung metrik pada business yang sedang diakses. `Dashboard::CustomersController` menggunakan aggregate COUNT/SUM/MAX per user.

Istilah perlu dijelaskan kepada klien:

- Nilai booking selesai adalah nilai layanan yang status booking-nya completed.

- Pembayaran online terkonfirmasi adalah PaymentAttempt yang memenuhi scope paid.

- Uang production mengecualikan sandbox dan riwayat Midtrans sandbox.

- Saldo owner/payout ke bank belum dihitung otomatis.

Grouping tanggal memakai waktu Jakarta agar booking dekat tengah malam masuk tanggal/bulan yang benar. Grouping bisnis menggunakan identitas ID; dua bisnis bernama sama tidak boleh dianggap satu.

CSV dibuat service `PaymentAttemptCsv`, termasuk perlindungan nilai yang dapat menjadi formula spreadsheet. Input pengguna yang diawali `=`, `+`, `-` atau `@` perlu diperlakukan sebagai data, bukan rumus. CSV berbeda dari file XLSX.

Saat menambah laporan: tulis arti metrik dahulu, tentukan timezone/status/periode, susun query, baru buat grafik. Grafik yang bagus dengan definisi salah tetap memberi informasi salah.

25 - Pengujian: bukti, bukan dugaan

Bab ini berisi perintah untuk Anda pelajari dan jalankan pada lingkungan test; penulisan dokumentasi ini tidak menjalankan suite aplikasi baru.

LATIHAN - contoh model spec pada katalog mini:

```
RSpec.describe Service, type: :model do
  it "menolak harga negatif" do
    service = Service.new(name: "Facial", price: -1, duration_minutes: 30)
    expect(service).not_to be_valid
  end
end
```

`describe` menyatakan objek yang diuji; `it` satu skenario; `expect` harapan. Untuk memakai contoh, tambahkan `rspec-rails` pada group development/test di Gemfile latihan, jalankan `bundle install`, lalu `bin/rails generate rspec:install`. Simpan contoh pada `spec/models/service_spec.rb`.

```
# Di project latihan atau database test terpisah
bundle exec rspec spec/models/service_spec.rb
```

Pada GlowBook, FactoryBot membuat user/bisnis/layanan yang memenuhi relasi. Request spec mengecek akses, respons dan perubahan data. Service spec mengecek logika booking/payment. Node tests mengecek JavaScript. Brakeman mencari pola keamanan; hasil nol peringatan bukan jaminan tidak ada bug.

Untuk GlowBook gunakan stack uji terpisah:

```
docker compose -f compose.test.yaml up --build --abort-on-container-exit --exit-code-from tests
docker compose -f compose.test.yaml down --remove-orphans --volumes
node --test spec/javascript/*_test.cjs
```

Pembersihan di sini hanya untuk project Compose verification yang bersifat sementara. Jangan menambahkan `--volumes` pada stack utama karena dapat menghapus volume database/upload utama.

Workflow CI juga memeriksa syntax/routes/templates/terjemahan, build production, konfigurasi HTTPS dan backup/restore. Baca `.github/workflows/ci.yml` untuk tahap aktual. Pengujian baru harus ditambahkan ketika perilaku baru dibuat; pengujian lama hijau tidak otomatis membuktikan semua fitur baru.

26 - Docker, deployment dan backup

Dockerfile memakai beberapa tahap: base, dependencies, build, verification dan image runtime. Runtime memakai user non-root. Build memasang library dan menghasilkan aset; runtime menjalankan aplikasi.

Compose utama menjalankan PostgreSQL, web dan worker. Web melakukan `db:prepare` pada entrypoint sebelum server mulai. Worker menunggu web sehat kemudian menjalankan `bin/jobs`.

`/up` adalah healthcheck, bukan uji seluruh fitur. Database healthy berarti dapat menerima koneksi, bukan berarti laporan atau pembayaran pasti benar.

KODE PROJECT - volume:

```
volumes:
  postgres_data:
  storage_data:
```

Volume mempertahankan data saat container dibuat ulang. Volume tetap dapat hilang karena penghapusan, kerusakan disk atau salah perintah. Karena itu backup tetap diperlukan.

Production membutuhkan APP_HOST HTTPS, secret baru, database, antrean, storage dan SMTP. Admin mengatur kredensial integrasi aplikasi, tetapi domain, DNS, HTTPS dan key utama tetap di server.

Tidak harus mengelola VPS sendiri: penyedia hosting aplikasi dapat menjalankan Rails. Konfigurasi Render/Railway belum disiapkan sebagai deployment terbukti pada snapshot ini. Jangan menganggap menghubungkan GitHub saja cukup: database queue, extension PostgreSQL `btree_gist`, port, worker, secret dan storage perlu diperiksa.

Gunakan `BACKUP_RESTORE.md`: backup primary DB, queue DB dan upload; simpan salinan di luar host; periksa checksum; lakukan restore pada lingkungan terpisah. Simpan key enkripsi terpisah dari backup. Sukses membuat dump belum membuktikan dump dapat dipulihkan.

Untuk admin production, gunakan task `glowbook:bootstrap` sesuai `SETUP.md`, bukan seed demo. Task mempertahankan akun/data yang sudah ada dan menolak menaikkan akun customer lama menjadi admin hanya karena email sama.

27 - Git dan pekerjaan sehari-hari

Git menyimpan perubahan; GitHub menyimpan remote repository. **Commit** adalah catatan lokal; **push** mengirim commit ke remote.

```
git status
git switch -c codex/latihan-deskripsi-layanan
git diff
git add app/models/service.rb app/views/dashboard/services/_form.html.erb
git commit -m "Add service description field"
git push -u origin codex/latihan-deskripsi-layanan
```

Contoh ini mengandaikan file sudah diubah dan branch latihan belum ada. Jangan menyalin contoh add jika perubahan Anda berada pada file berbeda. Gunakan `git diff` untuk membaca apa yang akan disimpan.

Untuk klien, satu perubahan yang jelas lebih mudah ditinjau daripada banyak fitur tak terkait pada satu commit. Jangan memasukkan `.env`, master key, dump, upload pelanggan atau data pribadi. Dua gambar lokal yang tidak terkait pembaruan GlowBook juga tidak otomatis dimasukkan ke commit.

Urutan kerja: pahami permintaan -> pilih file -> ubah secukupnya -> periksa diff -> verifikasi sesuai cakupan -> commit -> push -> pantau CI -> jelaskan hasil dan batasnya.

28 - Debugging tanpa panik

Gejala	Mulai periksa	Alasan
Halaman 404	Routes, parameter ID/slug/reference	URL mungkin menuju resource yang berbeda
Admin ditolak	Login/session dan role	Authentication dan authorization berbeda
Data tidak tersimpan	errors model, permit controller, constraint DB	Field form belum tentu diizinkan server
CSS tidak berubah	Source Tailwind, proses build, image yang berjalan	File source host tidak otomatis masuk image
Email belum datang	Worker, job gagal, SMTP, inbox/spam	Antrean bukan konfirmasi terkirim
Slot kosong	Jam buka, shift, break, block, kualifikasi, timezone	Availability menyaring banyak kondisi
Callback gagal	Mode/merchant/signature/amount/reference/status API	Redirect browser tidak dipercaya
Upload hilang	Storage service, volume/object storage	Filesystem container dapat sementara
Filter tidak cocok	Scope awal, query URL, basis tanggal	Tab/periode awal juga membatasi hasil
CI gagal	Tahap pertama yang gagal dan log terkait	Tahap berikutnya bisa hanya skipped

Prosedur sederhana: reproduksi, tulis hasil harapan vs aktual, baca log bagian error, cari file terkait dengan `rg`, ubah satu penyebab, lalu ulangi skenario. Jangan sekaligus mengganti dependency, database dan UI ketika penyebab belum jelas.

29 - Menjadi siap bekerja remote di Upwork

Kecepatan dengan AI dapat membantu, tetapi Anda tetap bertanggung jawab memahami perubahan, menjaga data klien dan menyerahkan hasil yang sesuai. Menghafal atau mengetik lebih lama bukan tujuan profesional.

Pada kontrak hourly, catat **waktu kerja yang benar-benar dilakukan**. Jangan menagih delapan jam bila kerja terkait kontrak hanya dua jam. Waktu analisis, implementasi, review, pengujian, debugging dan dokumentasi yang benar-benar dikerjakan dapat dijelaskan sebagai tugas; pastikan cakupannya disepakati klien. Belajar umum untuk meningkatkan kemampuan sendiri tidak otomatis menjadi jam tagihan klien.

Upwork menyediakan tracked time melalui Desktop App dan manual time bila diizinkan. Manual time tidak memperoleh Hourly Payment Protection yang sama seperti tracked time; periksa ketentuan resmi sebelum menggunakan. Penjelasan tracked/manual time.

Jika pekerjaan cepat dan scope jelas, kontrak fixed-price dengan milestone bisa dibicarakan sehingga nilai hasil disepakati di awal. Jangan mengubah model tagihan tanpa persetujuan klien. Ini saran cara menyusun pekerjaan, bukan janji pendapatan atau aturan universal semua klien.

Contoh catatan kerja yang jelas

```
Task: Add role filter to admin users table
Work: Reviewed existing access scope; implemented role options;
      checked owner/customer results; documented behavior.
Deliverable: commit + screenshots + validation results
Time: actual start/end recorded for this contract
```

Hindari memo "coding website" tanpa konteks. Jelaskan apa yang diselesaikan, file utama, hasil verifikasi, risiko dan pekerjaan tersisa.

AI sebagai tutor, bukan pengganti pemahaman

Minta AI: "jelaskan method ini", "beri satu latihan", "tunjukkan kemungkinan gagal", "review perubahan kecil ini". Sebelum menerima kode, jelaskan input, output, akses pengguna dan kegagalan yang mungkin terjadi. Ikuti kebijakan klien tentang penggunaan AI dan izin membagikan kode/data kepada layanan luar.

Milestone portofolio yang realistis

Milestone	Hasil yang dapat diperlihatkan
Katalog	CRUD layanan, validasi, tampilan responsive
Akses	Login dan pemisahan customer/owner/admin
Booking	Availability, transaksi, constraint dan skenario bentrok
Admin	Filter, pagination, pengelolaan dan laporan
Integrasi	Sandbox, email, callback dan dokumentasi setup
Rilis	CI, deployment, backup dan handover

Jangan menyatakan Anda menulis semuanya sendiri bila memakai bantuan. Anda dapat menjelaskan kontribusi dan kemampuan memelihara kode secara jujur.

30 - Rencana belajar delapan tahap

Ini urutan belajar yang disarankan, bukan target jam yang harus ditagihkan. Lama tiap tahap mengikuti kemampuan dan waktu Anda.

Tahap	Fokus	Tugas kecil
1	HTML, CSS, Ruby dasar	Buat kartu layanan dan kalkulator harga
2	MVC dan Active Record	Katalog layanan mini bab 09
3	Relasi dan Devise	Hubungkan bisnis-layanan dan batasi owner
4	Jadwal dan transaksi	Hitung slot satu staff, pelajari range
5	Booking dan akun	Simpan dua layanan; pelajari cancel/reschedule
6	Admin dan UX	Tambahkan satu filter dan perapian responsive
7	Integrasi dan pekerjaan belakang	Email test, chat, OAuth dan Duitku stub
8	Delivery remote	Git, CI, dokumentasi, demo dan handover

Selesai satu tahap ketika Anda bisa menjelaskan "request masuk ke mana, data diambil dari mana, siapa yang boleh mengubah, dan apa yang terjadi jika gagal".

31 - Cara membaca atlas kode

Atlas adalah pendamping, bukan bacaan dari halaman pertama sampai terakhir. Semua file sumber yang dimasukkan memiliki lokasi, isi bernomor baris, ringkasan tanggung jawab, serta catatan pola kode yang memang terdeteksi. Ringkasan pola tidak menggantikan review keamanan atau pembuktian perilaku.

Mulai dari satu jejak:

Pertanyaan	Jejak file
Bagaimana layanan dibuat?	routes -> dashboard/services controller -> Service -> form/view
Mengapa slot tidak tersedia?	bookings_flow -> BookingSequenceAvailability -> Availability -> BookingItem/constraint
Bagaimana admin mengedit user?	admin/users controller -> User -> admin/users/_form
Dari mana tabel mendapat filter?	Paginatable -> shared/_table_controls -> _table_pagination -> CSS
Bagaimana email terkirim?	BookingNotificationDelivery -> BookingMailer -> job -> ApplicationMailer -> worker config
Siapa boleh membaca chat?	ChatsController -> Conversation -> ChatMessage -> chat_controller.js
Kapan pembayaran lunas?	ProcessDuitkuNotification -> ReconcileDuitkuPayment -> DuitkuClient -> PaymentAttempt

Atlas mencakup sumber Ruby/ERB/JS/CSS, migration, konfigurasi terpilih, task, script deployment dan spec. Tidak menyertakan `.env.local`, master key, database dump, file upload, library pihak ketiga atau aset hasil build. File locale yang besar dijelaskan sebagai kelompok konfigurasi; versi mentahnya tetap tersedia dalam repository.

Untuk penjelasan satu baris yang belum Anda pahami, sebutkan nama file dan nomor baris atlas. Belajar lewat pertanyaan konkret jauh lebih mudah daripada meminta penjelasan seluruh aplikasi dalam satu sesi.

32 - Kamus istilah singkat

Istilah	Arti
CRUD	Create, Read, Update, Delete
ORM	Penghubung objek program dengan tabel database
Relation/scope	Query data yang dapat ditambah kondisi
Migration	Perubahan struktur database yang berurutan
Foreign key	Hubungan ID ke tabel induk
Validation	Pemeriksaan input sebelum disimpan
Constraint	Aturan yang ditegakkan database
Transaction	Beberapa perubahan menjadi satu kesatuan
Lock	Membatasi perubahan bersamaan pada data tertentu
Race condition	Hasil bergantung urutan request yang bersamaan
CSRF	Request tak diinginkan yang menyalahgunakan session pengguna
XSS	Konten berbahaya dieksekusi browser sebagai script
OAuth	Login/otorisasi melalui penyedia eksternal
Secret	Password/key yang harus dirahasiakan
Callback/webhook	Penyedia mengirim pemberitahuan ke endpoint aplikasi
Idempotent	Pengulangan tidak menggandakan efek yang sudah berhasil
Job/worker	Pekerjaan antrean/proses yang mengerjakannya
CI	Pemeriksaan otomatis atas commit
Deployment	Menjalankan versi aplikasi pada lingkungan tujuan
Backup/restore	Menyalin data/memulihkan data dari salinan

33 - Referensi dan dokumen lanjutan

Sumber utama panduan ini adalah kode lokal snapshot `cb1cc60`. Snippet project berasal dari file yang disebutkan; snippet latihan dibuat khusus untuk pembelajaran. Referensi resmi dibaca untuk membantu istilah dan ketentuan eksternal, bukan menyalin tutorial pihak lain.

- Ruby documentation - bahasa Ruby.

- Rails Guides - pilih versi yang sesuai ketika memeriksa API.

- Active Record API Rails 8.1 - query, model dan relasi.

- Turbo Handbook - konsep navigasi HTML.

- Stimulus Handbook - controller JavaScript.

- Devise repository - autentikasi.

- PostgreSQL range types - rentang dan overlap.

- Duitku POP dan status transaksi - API merchant.

- Upwork Time Tracker - pencatatan waktu aktual.

Dokumen operasional project: `SETUP.md`, `DUITKU_SETUP.md`, `BACKUP_RESTORE.md`, `ADMIN_FEATURES.md`, `RELEASE_READINESS.md`. Panduan belajar tidak menggantikan pemeriksaan production, dan klaim historis pengujian tidak berarti seluruh fitur sudah diuji lagi pada tanggal penulisan.

Langkah pertama setelah membaca: selesaikan katalog mini bab 09 tanpa meminta AI menulis seluruh fitur; gunakan AI untuk menjelaskan satu error atau satu method. Lalu kerjakan latihan 1-3 pada workbook.

Lampiran - Latihan dan Pembahasan

Gunakan bersama PANDUAN_GLOWBOOK.html. Semua perubahan eksperimen dilakukan pada `glowbook_latihan` atau salinan project dengan database latihan. Jangan gunakan data klien sebagai data belajar. Urutan ini tidak menetapkan jam kerja Upwork.

Cara mengerjakan

1. Baca tugas dan tulis rencana tiga kalimat.

2. Kerjakan dahulu sebelum membaca pembahasan.

3. Jelaskan input, proses, output dan kemungkinan kegagalan.

4. Simpan satu commit setelah tugas selesai dan diperiksa.

Latihan 1 - Menjelaskan satu request

Tugas: tulis urutan saat browser membuka `/admin/pengguna`.

Selesai jika: Anda menyebut routes, access guard, controller, model, filter dan view.

Pembahasan: GET dipetakan routes ke Admin::UsersController#index. Admin::BaseController menjalankan require_admin!. Scope User dipilih, Paginatable membaca kontrol tabel, PostgreSQL menyediakan data, lalu view index menghasilkan HTML. Pengguna yang tidak berhak tidak boleh mencapai data admin.

Latihan 2 - Ruby sederhana

Tugas: hitung harga tiga layanan `[100_000, 150_000, 75_000]`, lalu tampilkan totalnya.

Selesai jika: hasil 325000 dan Anda memahami Array/Integer.

```
harga = [100_000, 150_000, 75_000]
total = harga.sum
puts total
```

Pembahasan: underscore hanya membantu membaca angka. `sum` menjumlahkan elemen. `puts` menampilkan hasil; belum menyimpan apa pun ke database.

Latihan 3 - Katalog mini

Tugas: selesaikan CRUD mini bab 09: list, new dan create.

Selesai jika: layanan baru tampil setelah submit; nama kosong dan harga negatif ditolak; pesan error ditampilkan.

Pembahasan: route menentukan action, controller membuat Service dari strong parameters, model memvalidasi, view menampilkan hasil. Validasi form browser tidak menggantikan validasi server. Bila gagal, render new dengan status 422; bila berhasil, redirect ke daftar.

Latihan 4 - Tambah deskripsi

Tugas: tambahkan `description:text` pada katalog mini, tampilkan textarea dan deskripsinya di daftar.

Selesai jika: deskripsi tetap ada setelah halaman dimuat ulang.

Pembahasan: buat migration, migrate, permit `:description`, tambahkan `f.text_area`, lalu tampilkan `service.description`. Jangan hanya menambah field HTML: tanpa permit data tidak masuk ke model. Jangan

mengedit schema.rb secara langsung.

Latihan 5 - Relasi bisnis dan layanan

Tugas: rancang relasi Business-Service; jelaskan di tabel mana foreign key berada. Belum perlu menyalin seluruh schema GlowBook.

Selesai jika: Anda menjelaskan satu bisnis memiliki banyak layanan, service belongs_to business, dan services menyimpan business_id.

Pembahasan: `has_many :services` pada Business dan `belongs_to :business` pada Service. Migration membutuhkan reference business. Ketika menambah relasi pada data yang sudah ada, rencanakan pengisian foreign key lama sebelum memberi null:false; generator saja tidak menyelesaikan migrasi data.

Latihan 6 - Periksa akses owner

Tugas: jelaskan mengapa `@business.services.find(id)` lebih tepat daripada `Service.find(id)` untuk edit layanan owner.

Selesai jika: Anda menyebut scope, ownership dan kemungkinan mengganti ID pada URL.

Pembahasan: scope pertama membatasi record ke bisnis yang boleh diakses. Login owner tidak membuat semua layanan menjadi miliknya. Tombol edit disembunyikan bukan perlindungan server; endpoint juga perlu memeriksa akses.

Latihan 7 - Memahami interval

Tugas: staff punya booking 09.00-10.00. Apakah 09.30-10.30 bentrok? Apakah 10.00-10.30 bentrok jika range `[)`?

Jawaban: yang pertama bentrok; yang kedua tidak.

Pembahasan: `[)` memasukkan awal, mengecualikan akhir. Tugas lanjutan: tambahkan break 10.15-10.45. Walau tidak bentrok booking, layanan 10.00-10.30 tetap tidak tersedia karena break.

Latihan 8 - Rangkaian layanan

Tugas: layanan A 45 menit, B 30 menit, awal 09.00. Staff punya break 10.00-10.30. Tentukan waktu selesai dan apakah rangkaian tersedia.

Jawaban: akhir 10.15; tidak tersedia.

Pembahasan: A berakhir 09.45, B berakhir 10.15. Jangan hanya memeriksa apakah waktu awal 09.00 tersedia. Seluruh interval harus valid.

Latihan 9 - Transaction dan lock

Tugas: booking berhasil dibuat tetapi BookingItem kedua gagal. Jelaskan hasil bila keduanya berada dalam transaksi dan penyimpanan memakai bang method.

Jawaban: exception menyebabkan rollback perubahan dalam transaksi; jangan meninggalkan booking setengah lengkap.

Pembahasan: transaksi menjaga kesatuan perubahan. Lock mengatur perubahan bersamaan. Constraint overlap tetap diperlukan karena dua transaksi dapat semula sama-sama melihat slot kosong. Ketiganya mempunyai tugas berbeda.

Latihan 10 - Pagination

Tugas: 63 hasil, 25 per halaman. Berapa halaman? Berapa offset halaman ketiga? Apa hasil bila filter hanya menyisakan 6 record?

Jawaban: tiga halaman; offset 50; satu halaman untuk enam hasil.

Pembahasan: total halaman ceil(63/25). Halaman terakhir bukan selalu penuh. Page dari URL harus divalidasi/dibatasi. Filter harus diterapkan sebelum count, limit dan offset.

Latihan 11 - Filter tanggal

Tugas: jelaskan perbedaan filter created_at User dan start_time Booking.

Jawaban: yang pertama memilih tanggal akun dibuat; yang kedua tanggal janji layanan.

Pembahasan: jangan menyebut kedua filter sebagai "tanggal transaksi" tanpa konteks. GlowBook memberi label basis tanggal. Waktu harus sesuai timezone aplikasi.

Latihan 12 - Spacing CSS

Tugas: buat card filter dengan label tidak menempel tepi, pencarian selebar baris, dan dua field sejajar di layar kecil.

Selesai jika: padding sisi 16 px terlihat, label punya margin bawah, dan field tidak melebar keluar wadah.

Pembahasan: padding untuk sisi card, gap untuk jarak kolom, min-width:0 untuk grid item. Periksa view lebar kecil dan besar. Tidak perlu menambah JavaScript hanya untuk spacing.

Latihan 13 - Email dan chat

Tugas: booking berhasil tetapi email belum sampai. Buat daftar pemeriksaan. Jelaskan juga mengapa pesan chat memakai textContent.

Pembahasan: periksa konfigurasi SMTP aktif, antrean, worker, job gagal, penerima dan inbox/spam. `deliver_later` hanya mengantrekan. `textContent` menghindari menafsirkan isi pesan sebagai HTML yang dapat berbahaya.

Latihan 14 - Pembayaran palsu dari browser

Tugas: seseorang membuka return URL dengan `resultCode=00`. Bolehkah aplikasi langsung memberi status lunas?

Jawaban: tidak.

Pembahasan: browser dapat mengubah parameter. Pembayaran ditentukan oleh callback terverifikasi dan pemeriksaan API status. Cocokkan merchant, mode, order, reference dan nominal. Callback dua kali tidak boleh membuat efek pembayaran dua kali.

Latihan 15 - Bahasa

Tugas: tambahkan label fitur latihan dalam locale id dan en, lalu panggil t(...) pada view.

```
id:
  latihan:
    deskripsi: Deskripsi layanan

en:
  latihan:
    deskripsi: Service description
```

```
<%= t("latihan.deskripsi") %>
```

Pembahasan: key sama pada kedua file; value berbeda bahasa. Indentasi YAML menentukan hirarki. Tidak perlu mengganti nama key menjadi bahasa Inggris pada file en.

Latihan 16 - Uji dan debugging

Tugas: tambahkan model spec harga negatif pada katalog mini dan buat ia lulus karena model benar, bukan karena expectation dilemahkan.

Selesai jika: spec gagal ketika validasi sengaja dilepas dan berhasil ketika validasi dikembalikan.

Pembahasan: ini bukti spec menguji perilaku yang dimaksud. Skenario valid dan invalid sama-sama diperlukan; validasi yang menolak semua data juga dapat meluluskan satu spec invalid.

Latihan 17 - Deployment dan backup

Tugas: sebutkan tiga kelompok data yang perlu dipulihkan pada GlowBook production dan satu secret yang harus dipertahankan.

Jawaban: database aplikasi, database queue, file upload; key utama SECRET_KEY_BASE untuk membaca konfigurasi terenkripsi.

Pembahasan: metadata Active Storage di database tidak menggantikan byte file. Backup key perlu disimpan aman terpisah. Restore ke lingkungan terpisah untuk menghindari menimpa data aktif.

Latihan 18 - Komunikasi klien

Tugas: tulis update singkat untuk fitur filter pengguna. Sertakan hasil, bukti, keterbatasan dan actual time bila kontrak hourly.

Contoh:

```
Implemented admin user search, role filter and page-size controls.
Verified 10-row paging and combined role/email search.
Commit: [your actual commit]
Known limit: All records can be expensive on large datasets.
Time: actual work recorded under the agreed contract scope.
```

Pembahasan: jangan menulis jumlah jam yang tidak dikerjakan. Jangan menyebut "all tests pass" bila hanya membuka satu halaman. Dokumentasi, review dan pengujian nyata merupakan hasil kerja yang bisa dijelaskan dengan konkret.

Penilaian diri

Pertanyaan	Jawab Ya/Belum
Saya dapat menelusuri satu URL sampai view.	
Saya memahami perbedaan model validation dan DB constraint.	
Saya dapat membuat CRUD kecil tanpa generator penuh.	
Saya dapat menjelaskan scope owner dan strong parameters.	
Saya tahu mengapa booking memakai range dan transaksi.	
Saya tidak memakai return URL untuk memutuskan pembayaran.	
Saya tahu kapan perlu worker dan permanent storage.	
Saya dapat membaca diff dan membuat commit yang jelas.	
Saya dapat melaporkan hasil tanpa melebih-lebihkan bukti.	

Jika banyak jawaban Belum, pilih satu latihan dan ulangi. Jangan menambah semua teknologi sekaligus. Pada sesi belajar berikutnya, mulai dari bab 09 dan satu error yang dapat direproduksi.